Lecture 10: Ensemble models

Machine Learning 2026

Federico Chiariotti (federico.chiariotti@unipd.it)



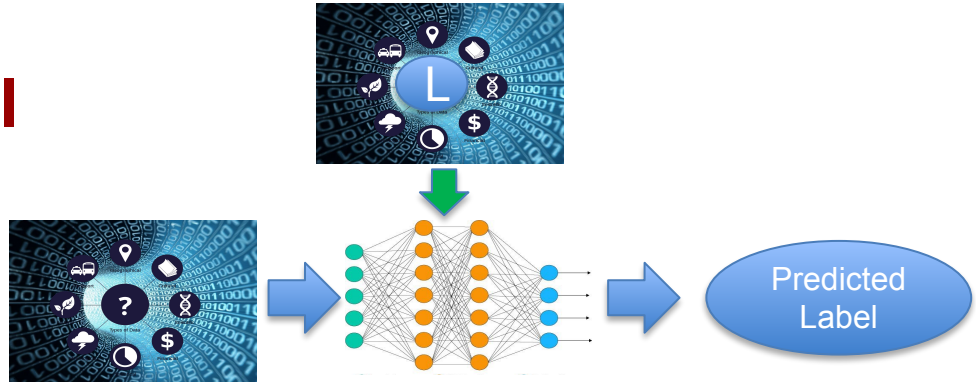
Università degli Studi di Padova

Recap



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Supervised learning model



- Prediction rule $h : X \rightarrow Y$
  - The learner's output (hypothesis)
  - $A(S)$: hypothesis produced by algorithm A when it is fed training set S
- Data generation model
  - D is a distribution over X **(unknown to the machine learning algorithm)**
  - Instances are labeled according to $f : X \rightarrow Y$ **(unknown to the ML algorithm)**
  - Training set: sampling according to $D, y_i = f(x_i)\ \forall x_i \in S$
- Success metric
  - Classifier error: probability of predicting the wrong label over D

Empirical Risk Minimization (ERM)

- ➔ Learner outputs $h_S : X \to Y$
- ➔ **Goal: find $\mathbf{h}^*$ that minimizes $\mathbf{L_{D,f}(h)}$**
- ➔ Both D and f are unknown!

ERM: we minimize the loss on the training set $L_S(h) = \frac{\sum_{i=1}^{m} |h(x_i) - y_i|}{m}$

This works if we use a 0-1 loss: the definition can be generalized

The empirical risk is also called training error or training loss

No free lunch theorem

Let A be any learning algorithm for binary classification, with 0-1 loss, over a domain X. Let m be any number smaller than $\frac{|X|}{2}$. There exists a distribution D over $X \times \{0,1\}$ such that:

1. There exists a function $f : X \to \{0,1\}$ with $L_D(f) = 0$
2. We have $L_D(A(S) \geq \frac{1}{8}$ with probability $\frac{1}{7}$ over the choice of $S \sim D^m$

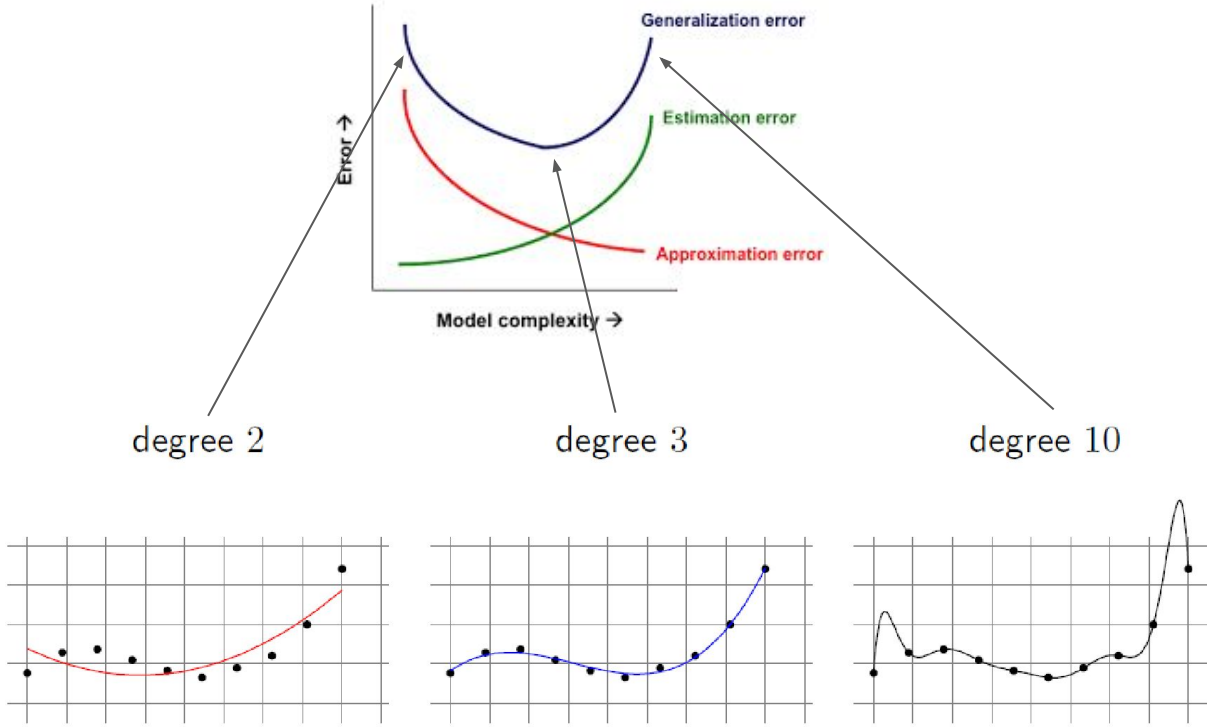
Corollary: if H is the set of all functions and X is an infinite set, we do **not** have PAC learnability

The fundamental theorem of PAC learning

Let H be a hypothesis class for a binary classification problem with 0-1 loss. The following statements are equivalent:

1. H has the uniform convergence property
2. Any ERM rule is a successful agnostic PAC learner for H
3. H is agnostic PAC learnable
4. H is PAC learnable (under realizability condition)
5. Any ERM rule is a successful PAC learner for H (under realizability condition)
6. H has a finite VC dimension

Example: polynomial fitting



Primal and dual problems

We can define a Lagrangian dual problem by turning a constrained problem into an unconstrained one.

Each constraint becomes a penalty in the objective function, with an associated Lagrange multiplier

The solution to the dual problem is *better* than the solution to the primal problem (but it might not be valid)



Convex programming

A problem is convex if we have two conditions:

- The objective function is convex
- The feasible set is convex
  - Inequality constraints should be convex
  - Equality constraints should be affine functions

We can solve convex problems efficiently!

Example: primal and dual problems

Problem: minimize $f(x, y) = (x + y)^2$ such that $x^2 + y^2 \leq 1$

We move to the unconstrained problem

$$f'(x, y) = (x + y)^2 + \lambda(x^2 + y^2 - 1)$$

This is still a convex function! We can find the minimum by setting the gradient to 0. By gradually increasing the multiplier, we get closer and closer to the primal problem

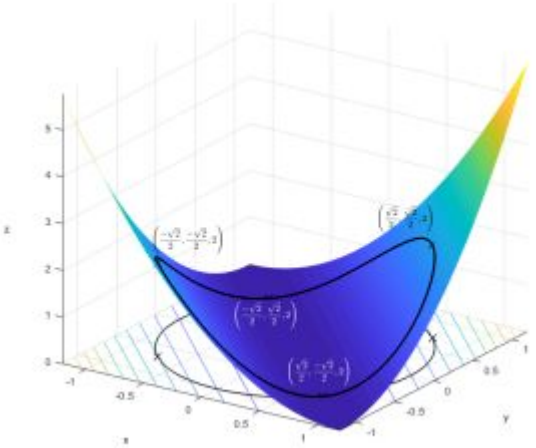


Image: Wikimedia Foundation

Using the gradient

Since we have a single minimum, how about trying to follow the gradient?

This is something like a ball falling down a curved surface and coming to rest on the lowest point

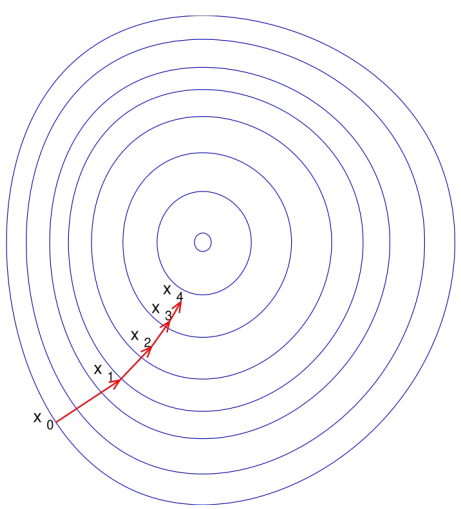


Image: Wikimedia Foundation

Gradient descent

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_d} \right)$$

The gradient points towards the direction of steepest ascent, it corresponds to the first-order Taylor approximation of the function

The approximation works for small steps, but breaks down over larger ones

Gradient descent algorithm

GD algorithm:

$\boldsymbol{w}^{(0)} \leftarrow \mathbf{0}$

for $t \leftarrow 0$ to $T - 1$ do

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \nabla f(\boldsymbol{w}^{(t)})$$

return $\bar{\boldsymbol{w}} = \boldsymbol{w}^{(T)}$

Learning rate
(controls the size of the steps)

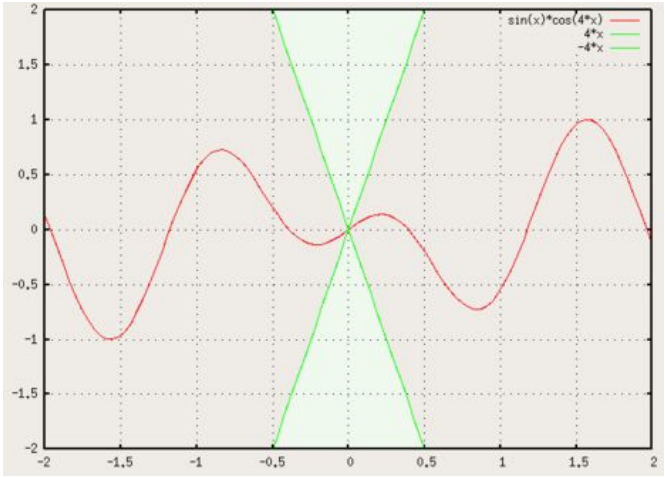
We stop when the gradient is 0 (minimum) or when we reach a maximum number of iterations

Gradient descent: convergence

If the target function is convex and ρ-Lipschitz, and we run GD for T steps, we have:

$$\eta = \sqrt{\frac{B^2}{\rho^2 T}} \Rightarrow f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \frac{B\rho}{\sqrt{T}}$$

Upper bound on $||\mathbf{w}^*||$



Proof: not part of the course (see Sec. 14.1.1 in the book)

A simple corollary

If the target function is convex and ρ-Lipschitz, and we run GD for T steps, we have:

$$\eta = \sqrt{\frac{B^2}{\rho^2 T}} \Rightarrow f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \frac{B\rho}{\sqrt{T}}$$

If we want a precision ε, we can run the GD algorithm over enough steps.

$$T \geq \frac{B^2\rho^2}{\varepsilon^2} \Rightarrow f(\bar{\mathbf{w}}) - f(\mathbf{w}^*) \leq \varepsilon$$

This is not learning: the error bound is with probability 1!

Ensemble models

Ensemble models rely on a simple principle: different learners may capture different aspects of the problem, and combining multiple learners may increase their power

If we can add some randomness to individual model, we can avoid overfitting and reduce computational complexity: multiple simple model can be more robust than a single complex one!

Part 1:
Weak learnability and boosting



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Ensemble models

Ensemble models rely on a simple principle: different learners may capture different aspects of the problem, and combining multiple learners may increase their power

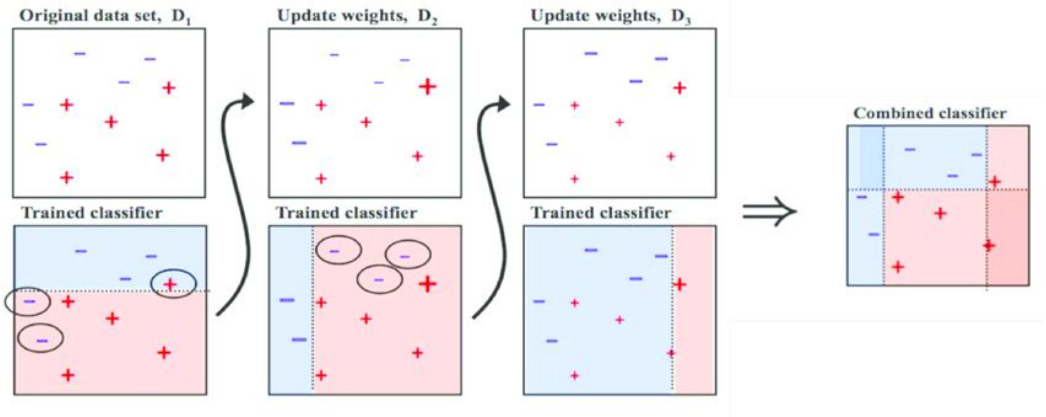
If we can add some randomness to individual model, we can avoid overfitting and reduce computational complexity: multiple simple model can be more robust than a single complex one!

Boosting

Boosting is another tool to balance the bias-complexity trade-off

➔ It also reduces computational complexity!

The basic idea is to aggregate multiple *weak* learners into a *strong* learner (i.e., a linear combination of simple hypotheses that can represent complex functions)



Weak and strong learnability

A hypothesis class H is PAC learnable if there is a function $m_H : (0,1)^2 \to \mathbb{N}$ and a learning algorithm such that for every $\delta, \varepsilon \in (0,1)$ and for every learning problem D, f with the realizability assumption, the algorithm will satisfy the PAC condition over an IID training set of size $m \geq m_H$

A hypothesis class H is **γ-weak** learnable if there is a function $m_H : (0,1)^2 \to \mathbb{N}$ and a learning algorithm such that for every $\delta \in (0,1)$ and for every learning problem D, f with the realizability assumption, the algorithm will satisfy the following condition over an IID training set of size $m \geq m_H$:

$$L_{D,f}(h) \leq \frac{1}{2} - \gamma$$

Weak and strong learnability

A hypothesis class H is **γ-weak** learnable if there is a function $m_H : (0,1)^2 \to \mathbb{N}$ and a learning algorithm such that for every $\delta \in (0,1)$ and for every learning problem D, f with the realizability assumption, the algorithm will satisfy the following condition over an IID training set of size $m \geq m_H$:

$$L_{D,f}(h) \leq \frac{1}{2} - \gamma$$

As the name suggests, weak learnability is a much weaker hypothesis: we just need to find a hypothesis with $L_{D,f}(h) \leq \frac{1}{2} - \gamma$, without worrying about arbitrarily small errors – in fact, this is only slightly better than a random guess!

The fundamental theorem of PAC learning

Let H be a hypothesis class for a binary classification problem with 0-1 loss. The following statements are equivalent:

1. H has the uniform convergence property
2. Any ERM rule is a successful agnostic PAC learner for H
3. H is agnostic PAC learnable
4. H is PAC learnable (under realizability condition)
5. Any ERM rule is a successful PAC learner for H (under realizability condition)
6. H has a finite VC dimension

Applies to weak learners as well!

Weak and strong learnability

In terms of sample complexity, we are still bound by the quantitative version of the fundamental theorem

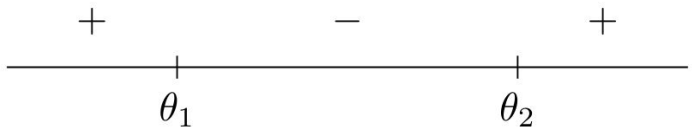
➔ Weak learners are simple, and some can be implemented efficiently
➔ We can use ERM over a simple hypothesis class B

However, this poses two questions:

1. Can we boost an efficient weak learner into an efficient strong learner?
2. What hypothesis class can we use?

Proof: decision stumps are weak learners

$$B = \{x \to b\text{sign}(x - \theta), b \in \{-1, 1\}, \theta \in \mathbb{R}\}$$



1. Every classifier in H consists of 3 regions
2. For any pair of regions, there is a stump that agrees exactly
3. At least one pair of regions must contain ⅔ of the points
4. There is a decision stump with error at most ⅓
5. The VC-dim. of the stump is 2, so it is learnable by agnostic PAC!

Efficient ERM for decision stumps

Base hypothesis class: $B = \{x \to b\text{sign}(x - \theta), b \in \{-1, 1\}, \theta \in \mathbb{R}\}$

Let us assume that $b = 1$

We define a probability mass function (PMF) $\mathbf{d}$ over the training set and

$$L_{\mathbf{d}}(h) = \sum_{i=1}^{m} d_i I(h(x_i) \neq y_i)$$

This is a generalization of empirical risk (which uses a uniform PMF)

Efficient ERM for decision stumps

$$L_{\mathbf{d}}(h) = \sum_{i=1}^{m} d_i I(h(x_i) \neq y_i)$$

Each decision stump picks an index j and a threshold θ. The solution is

$$\min_{j \in \{1,\dots,n\}} \min_{\theta \in \mathbb{R}} \left(\sum_{i:y_i=1} d_i I(x_{i,j} \leq \theta) + \sum_{i:y_i=-1} d_i I(x_{i,j} > \theta) \right)$$

The only points that matter are the ones in the training set: we define

$$\Theta_j = \left\{ \frac{x_{i,j} + x_{k,j}}{2}, i, k : x_{i,j} < x_{k,j}, x_{ell,j} \notin [x_{i,j}, x_{k,j}] \forall \ell \notin \{i, k\} \right\}$$

There are at most m-1 points: we can brute force the solution in O(mn)

ERM for decision stumps: pseudocode

ERM for Decision Stumps

input:

training set $S = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$

distribution vector $\mathbf{D}$

goal: Find $j^\star, \theta^\star$ that solve Equation (10.1)

initialize: $F^\star = \infty$

for $j = 1, \ldots, d$

sort S using the j'th coordinate, and denote

$x_{1,j} \leq x_{2,j} \leq \cdots \leq x_{m,j} \leq x_{m+1,j} \stackrel{\text{def}}{=} x_{m,j} + 1$

$F = \sum_{i:y_i=1} D_i$

if $F < F^\star$

$F^\star = F$, $\theta^\star = x_{1,j} - 1$, $j^\star = j$

for $i = 1, \ldots, m$

$F = F - y_i D_i$

if $F < F^\star$ and $x_{i,j} \neq x_{i+1,j}$

$F^\star = F$, $\theta^\star = \frac{1}{2}(x_{i,j} + x_{i+1,j})$, $j^\star = j$

output $j^\star, \theta^\star$

AdaBoost

AdaBoost was first presented in 1995 by Freund and Schapire, who won a Gödel prize for it in 2003

It combines weak learners to create a stronger model (but can also use more powerful methods, such as decision trees, as a base algorithm). The weak learner has an error (with probability at least $1 - \delta$):

$$\varepsilon_t = L_{\mathbf{d}}(h_t) = \sum_{i=1}^{m} d_i I(h_t(x_i) \neq y_i) \leq \frac{1}{2} - \gamma$$

AdaBoost gives h_t a weight $\frac{1}{2}\log\left(\frac{1}{\varepsilon_t} - 1\right)$

Boosting

After each round, we change $\mathbf{d}$ to give a higher weight to misclassified samples, using a softmax distribution

At each round, we boost towards a better hypothesis by adding another weak learner

AdaBoost

input:

training set $S = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$

weak learner WL

number of rounds T

initialize $\mathbf{D}^{(1)} = (\frac{1}{m}, \ldots, \frac{1}{m})$.

for $t = 1, \ldots, T$:

invoke weak learner $h_t = \mathrm{WL}(\mathbf{D}^{(t)}, S)$

compute $\epsilon_t = \sum_{i=1}^{m} D_i^{(t)} \mathbb{1}_{[y_i \neq h_t(\mathbf{x}_i)]}$

let $w_t = \frac{1}{2} \log\left(\frac{1}{\epsilon_t} - 1\right)$

update $D_i^{(t+1)} = \frac{D_i^{(t)} \exp(-w_t y_i h_t(\mathbf{x}_i))}{\sum_{j=1}^{m} D_j^{(t)} \exp(-w_t y_j h_t(\mathbf{x}_j))}$ for all $i = 1, \ldots, m$

output the hypothesis $h_s(\mathbf{x}) = \mathrm{sign}\left(\sum_{t=1}^{T} w_t h_t(\mathbf{x})\right)$.

AdaBoost convergence

If S is a training set and W is a γ-weak learner, the training error of the AdaBoost hypothesis after T iterations is at most

$$L_S(h_S) = \frac{1}{m}\sum_{i=1}^{m} I(h_S(x_i) \neq y_i) \leq e^{-2\gamma^2 T}$$

The proof is not part of the course, but it is in chapter 10.2 of the book

AdaBoost: empirical and true risk

If S is a training set and W is a γ-weak learner, the training error of the AdaBoost hypothesis after T iterations is at most

$$L_S(h_S) = \frac{1}{m}\sum_{i=1}^{m} I(h_S(x_i) \neq y_i) \leq e^{-2\gamma^2 T}$$

What about the true risk? We need to consider the boosted hypothesis as a linear combination of base hypotheses:

$$L(B,T) = \left\{ x \to \text{sign}\left(\sum_{t=1}^{T} w_t h_t(x)\right) : \mathbf{w} \in \mathbb{R}^T, h_t \in B \forall t \right\}.$$

The VC dimension of boosted solutions

If B is a base class and $L(B,T)$ is a boosted solution, with $\mathrm{VC}(B) \geq 3$ and $T \geq 3$, we have

$$\mathrm{VC}(L(B,T)) \leq T(\mathrm{VC}(B)+1)(3\log(T(\mathrm{VC}(B)+1))+2)$$

We can use the number of boosting iterations T to tune the bias-complexity trade-off: as T grows, the empirical risk goes to 0, but the estimation error grows. In other words, boosting increases the complexity of the hypothesis class, increasing its power but also the risk of overfitting

Proof: first step

$C = \{x_1, \ldots, x_m\}$ is a set that can be shattered by $L(B, T)$

1. We choose $h_1, \ldots, h_T \in B$
2. We apply a halfspace hypothesis $\mathbf{w}$

Sauer's lemma:

Let H be a class with $\mathrm{VC}(H) \leq d < \infty$. The number $\tau_H(m)$ of different possible restrictions over a set of size $m > d + 1$ is bounded by:

$$\tau_H(m) \leq \left(\frac{em}{d}\right)^d$$

Proof: second step

By Sauer's lemma, there are at most $\left(\frac{em}{d}\right)^d$ different labelings obtained from B over C

We need to choose T out of $\left(\frac{em}{d}\right)^d$ hypotheses

There are at most $\left(\frac{em}{d}\right)^{dT}$ possible choices

Proof: third step

When we apply the linear predictor, we can use Sauer's lemma again: there are at most $\left(\frac{em}{T}\right)^T$ different dichotomies (the VC dimension of the halfspace hypothesis class is equal to the dimension in homogeneous coordinates)

The overall number of dichotomies is bounded by:

$$\tau_{L(B,T)}(m) \leq \left(\frac{em}{d}\right)^{dT} \left(\frac{em}{T}\right)^T$$

Since $T \geq 3$ and $d \geq 3$, we have $\frac{e}{T} < 1$ and $\frac{e}{d} < 1$: we have

$$\left(\frac{em}{d}\right)^{dT} \left(\frac{em}{T}\right)^T < m^{(d+1)T}$$

Proof: third step

We have fewer than $m^{(d+1)T}$ different functions, but we need to have 2^m to shatter the set:

$$m^{(d+1)T} \geq 2^m$$

We can turn it around to get:

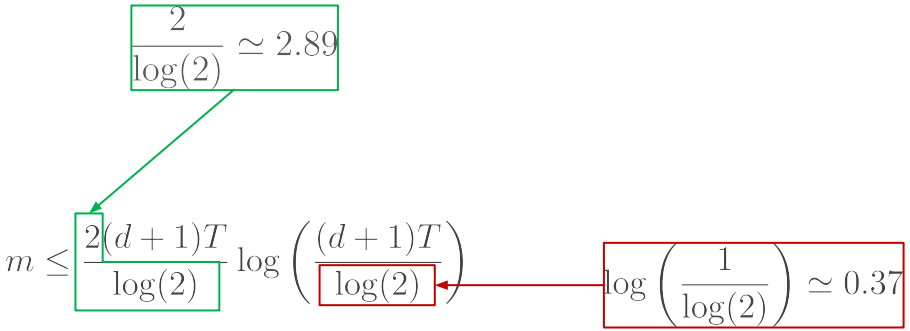
$$m \leq \log(m) \frac{(d+1)T}{\log(2)}$$

We know that $x < a\log(x) \implies x < 2a\log(a)$ (see Appendix A in the book)

$$m \leq \frac{2(d+1)T}{\log(2)} \log\left(\frac{(d+1)T}{\log(2)}\right)$$

Proof: final step

We know that $\log(2) \simeq 0.69$



We can just substitute 2 and 3 for the parts to get a looser bound with whole numbers:

$$m \leq (d+1)T(3\log((d+1)T) + 2)$$

This is the theorem statement.

Gradient boosting

Let us consider regression (the same thing works for classification)

We start from a simple model F_1, and gradually add models to boost it: we train a new model h_m on the residual $y - F_m(x)$

$$F_{m+1}(x) = F_m(x) + h_m(x)$$

The residual is equivalent to the negative gradient over the loss:

$$L_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}(y_i - F_m(x_i))^2$$

$$-\frac{\partial L_{\text{MSE}}}{\partial F_m(x_i)} = \frac{2}{m}(y_i - F_m(x_i))$$

Gradient boosting

Each round of gradient boosting then follows 3 steps:

1. Compute the pseudo-residual $r_m(x_i) = -\frac{\partial L_{\text{MSE}}}{\partial F_m(x_i)}$ for all points in the training set

2. Fit $h_m(x)$ to the pseudo-residual using a weak learner (usually a decision tree)

3. Solve the following optimization problem:

$$\gamma_m = \arg\min_\gamma \sum_{i=1}^{m} \left(F_m(x_i) + \gamma h_m(x_i) - y_i\right)^2$$

Boosting too much can also result in overfitting!

XGBoost

eXtreme Gradient Boosting (XGBoost) is currently the golden standard boosting algorithm. It uses a second-degree Taylor approximation: we compute the gradient and Hessian

$$\hat{g}_m(x_i) = \frac{\partial \ell(F_m(x_i), y_i)}{\partial F_m(x_i)} \qquad \hat{h}_m(x_i) = \frac{\partial^2 \ell(F_m(x_i), y_i)}{\partial F_m(x_i)^2}$$

We then fit the weak learner to $\frac{\hat{g}(x_i)}{\hat{h}(x_i)}$

XGBoost also usually implements regularization. Ways to avoid overfitting include reducing the number of estimators and early stopping

Part 2: Decision trees and random forests

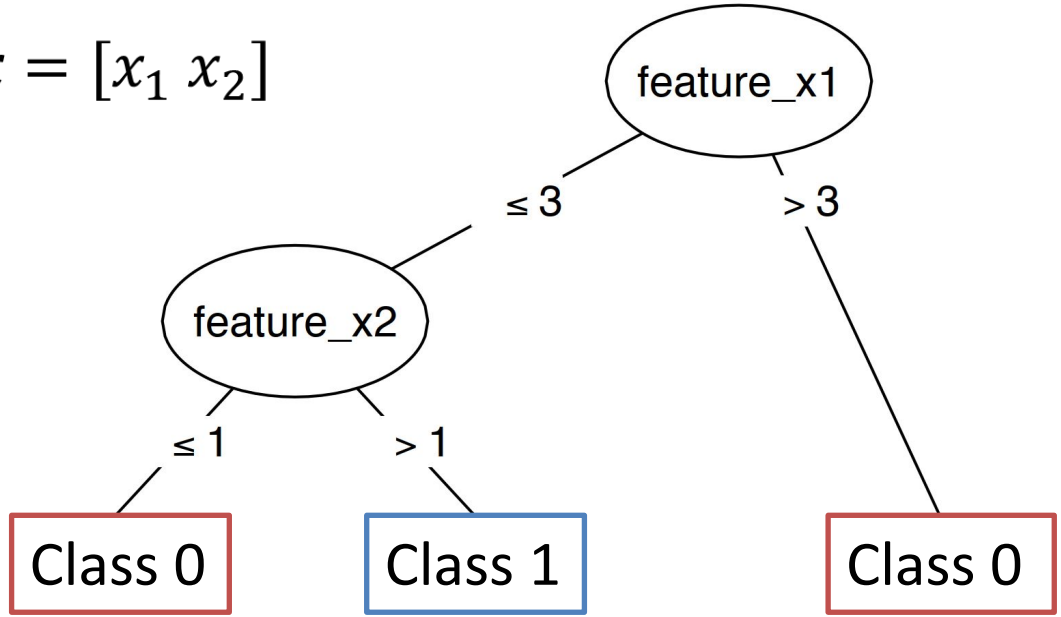


UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Decision trees

Decision trees are a sequence of decision stumps

$$\boldsymbol{x} = [x_1\ x_2]$$



The ID3 algorithm

Iterative dichotomizer algorithm:

1. Run a decision stump
2. Create another decision stump for each subgroup
3. Stop when all points in a subgroup have the same label

We can split multiple times over the same feature if it is real

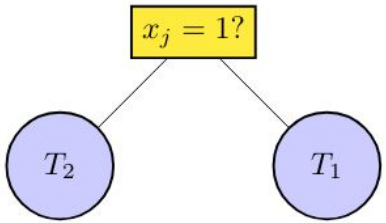
The ID3 algorithm

This version assumes binary features (which can only be split once)

Information gain

$\texttt{ID3}(S, A)$

INPUT: training set S, feature subset $A \subseteq [d]$
if all examples in S are labeled by 1, return a leaf 1
if all examples in S are labeled by 0, return a leaf 0
if $A = \emptyset$, return a leaf whose value = majority of labels in S
else :
  Let $j = \text{argmax}_{i \in A}$ $\texttt{Gain}(S, i)$
  if all examples in S have the same label
    Return a leaf whose value = majority of labels in S
  else
    Let T_1 be the tree returned by $\texttt{ID3}(\{(\mathbf{x}, y) \in S : x_j = 1\}, A \setminus \{j\})$.
    Let T_2 be the tree returned by $\texttt{ID3}(\{(\mathbf{x}, y) \in S : x_j = 0\}, A \setminus \{j\})$.
    Return the tree:



Information gain

The entropy determines the randomness of the classification in a set, i.e., how high the uncertainty is over the class

$$H(n_1, n_{-1}) = -\frac{n_1}{n_1 + n_{-1}} \log_2 \left(\frac{n_1}{n_1 + n_{-1}} \right) - \frac{n_{-1}}{n_1 + n_{-1}} \log_2 \left(\frac{n_{-1}}{n_1 + n_{-1}} \right)$$

To evaluate a split, we compute the conditional entropy:

$$H(\ell, r) = \frac{n_\ell}{n_\ell + n_r} H(n_1^\ell, n_{-1}^\ell) + \frac{n_r}{n_\ell + n_r} H(n_1^r, n_{-1}^r)$$

ID3 then tries to find the left-right split that minimizes the overall entropy

Other gain metrics for decision trees

Training accuracy: considering tree T, we get

$$G(T) = \frac{1}{m} \sum_{i=1}^{m} I(T(x_i) = y_i)$$

We can also use the Gini index:

$$G(T) = \frac{\left(\sum_{i=1}^{m} I(T(x_i) = 1)\right)\left(\sum_{i=1}^{m} I(T(x_i) = -1)\right)}{2m^2}$$

The Gini index measures the imbalance between classes (kind of like information gain), it represents a smooth and concave upper bound to the training error

Preventing overfitting

A tree of depth d has a VC dimension 2^d, as it can represent that many possible different functions

ID3 will always reach 0 empirical risk, as it does not set a limit on the depth of the tree, but it will overfit **a lot**: by definition, it will shatter the training set!

What we can do is reduce the complexity, going back to Minimum Description Length regularization!

Minimum Description Length

Let us put every hypothesis in a singleton class. By Hoeffding's inequality, this class has uniform convergence with $m^{\mathrm{UC}}(\varepsilon, \delta) = \frac{\log(2/\delta)}{\varepsilon^2}$. We then get

$$\varepsilon_h(m, \delta) = \sqrt{\frac{\log(2/\delta)}{2m}}$$

Suppose we have a way of describing a hypothesis using a prefix code. We use the length of the description as a weight:

$$h_{\mathrm{MDL}} = \arg\min_{h \in H} L_S(h) + \sqrt{\frac{|h| + \log(2/\delta)}{2m}}$$

A description language for decision trees

Each node may be:

1. An internal node for feature j
2. A leaf (class -1)
3. A leaf (class 1)
4. End of the code

We have (n+3) options for each node, so we can encode a tree with v nodes using $(v+1)\log_2(n+3)$ bits. We can state the bound:

$$L_D(h) \leq L_S(h) + \sqrt{\frac{(v+1)\log_2(n+3) + log(2/\delta)}{2m}}$$

Pruning

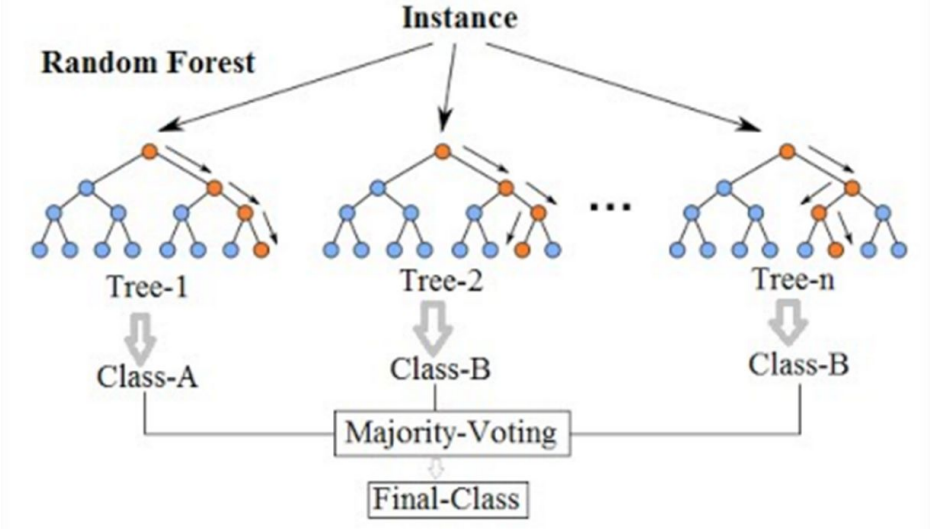
Generic Tree Pruning Procedure

input:
function $f(T, m)$ (bound/estimate for the generalization error of a decision tree T, based on a sample of size m),
tree T.

foreach node j in a bottom-up walk on T (from leaves to root):
find T' which minimizes $f(T', m)$, where T' is any of the following:
the current tree after replacing node j with a leaf 1.
the current tree after replacing node j with a leaf 0.
the current tree after replacing node j with its left subtree.
the current tree after replacing node j with its right subtree.
the current tree.
let $T := T'$.

Random forests

We can follow a similar procedure to boosting, but consider multiple decision trees: Salzberg and Heath introduced the concept of a *random forest* in 1993. A random forests consists of many individual trees with a degree of randomness, with the outcome determined by majority voting



Constructing trees: bagging

First, we increase randomness by repeating features: we can sample a new training set S' of size m' by sampling uniformly **with replacement** over S

Example: $S = \{1, 2, 3, 4, 5, 6\}$

$S' = \{1, 2, 1, 4, 6, 6, 5\}$

$S'' = \{2, 3, 3, 3, 4, 1, 6\}$

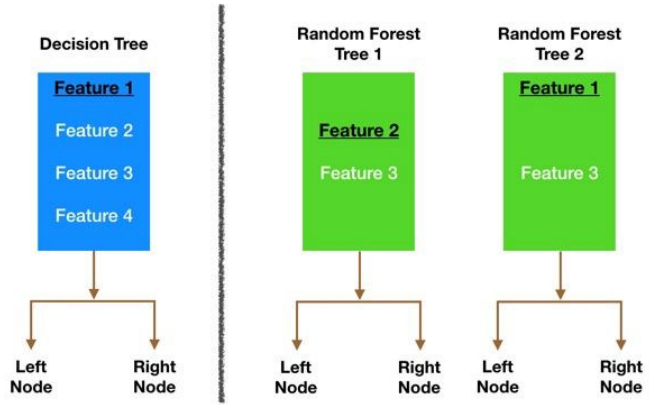
This changes the weight of each feature in the gain, resulting in slightly different trees

Out-of-bag error can be used as a validation score

Constructing trees: random subspace projection

The second step is to select random subsets of features for each tree

This increases randomization and reduces overfitting



Extra trees classification

Extra trees is a classification method with a different randomization strategy: instead of using bagging, it trains over the whole datasets, but randomly selects the threshold for each feature

It obtains similar performance to random forests, but is computationally lighter

Part 3: Exercises

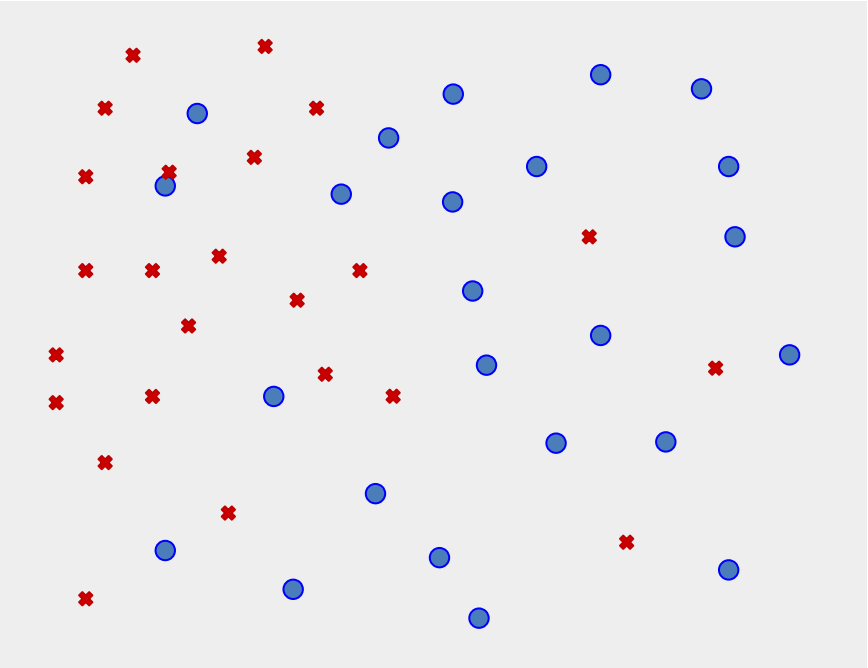


UNIVERSITÀ
DEGLI STUDI
DI PADOVA

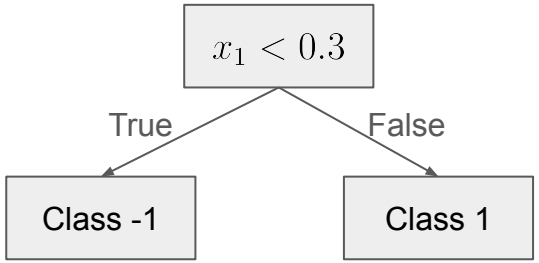
Exercise: training a tree with ID3

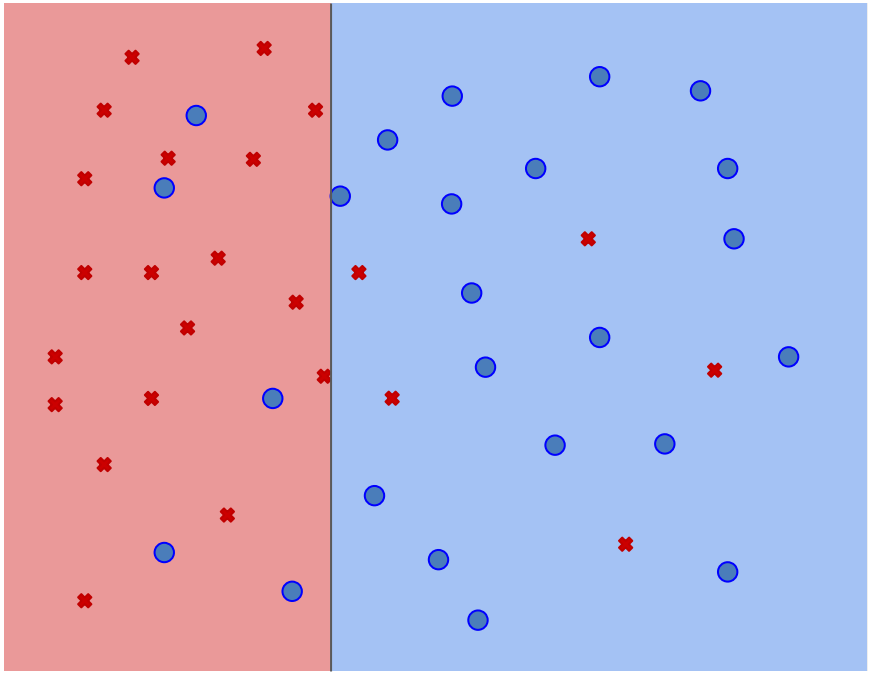
We use the information gain as a metric

There are 24 crosses and 24 circles.



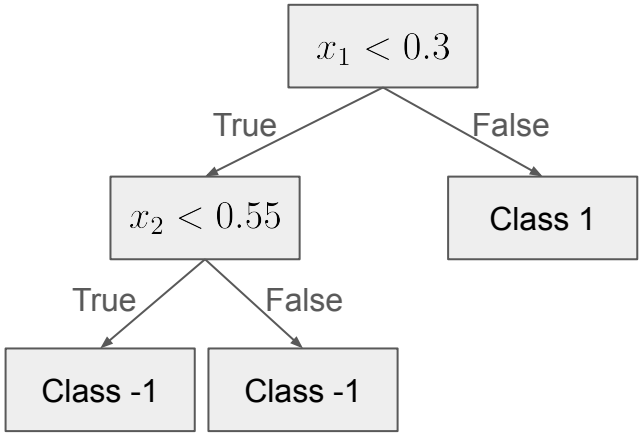
First division

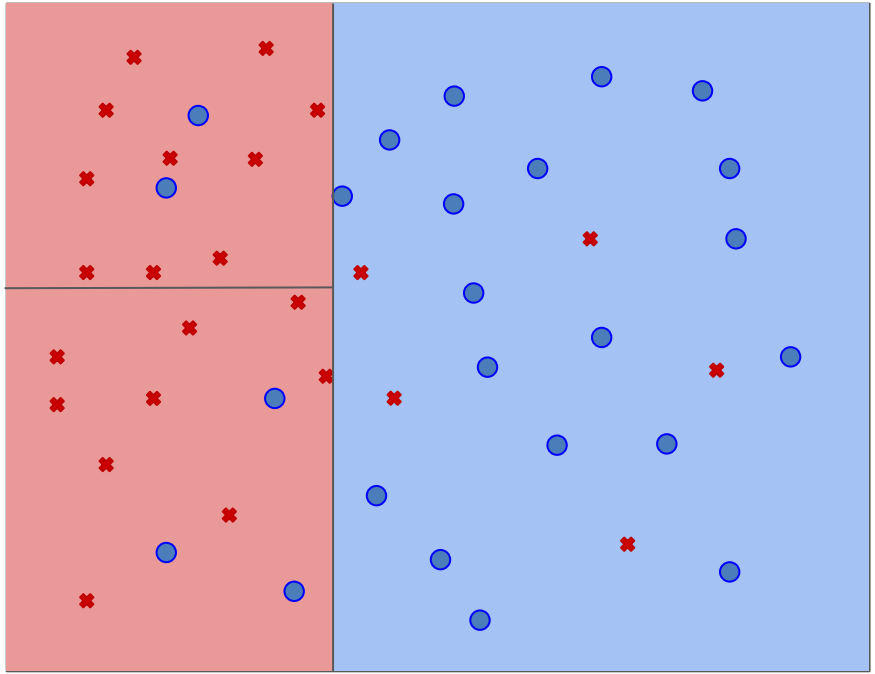




Left subtree

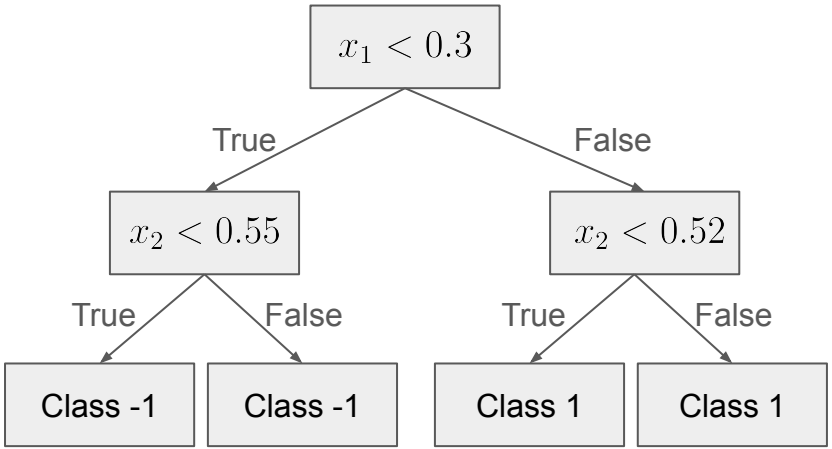
Both groups belong to class -1!

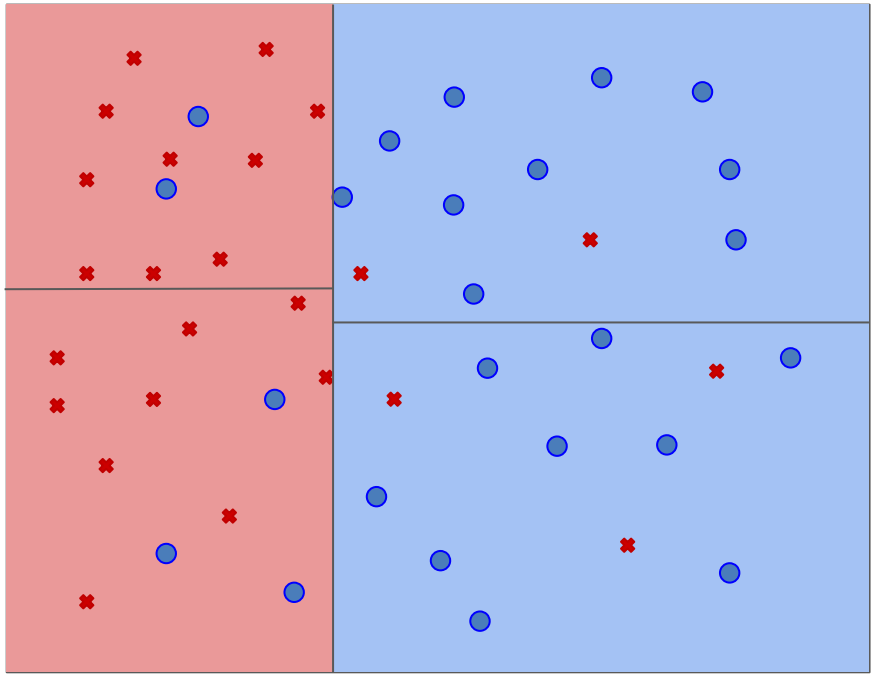




Right subtree

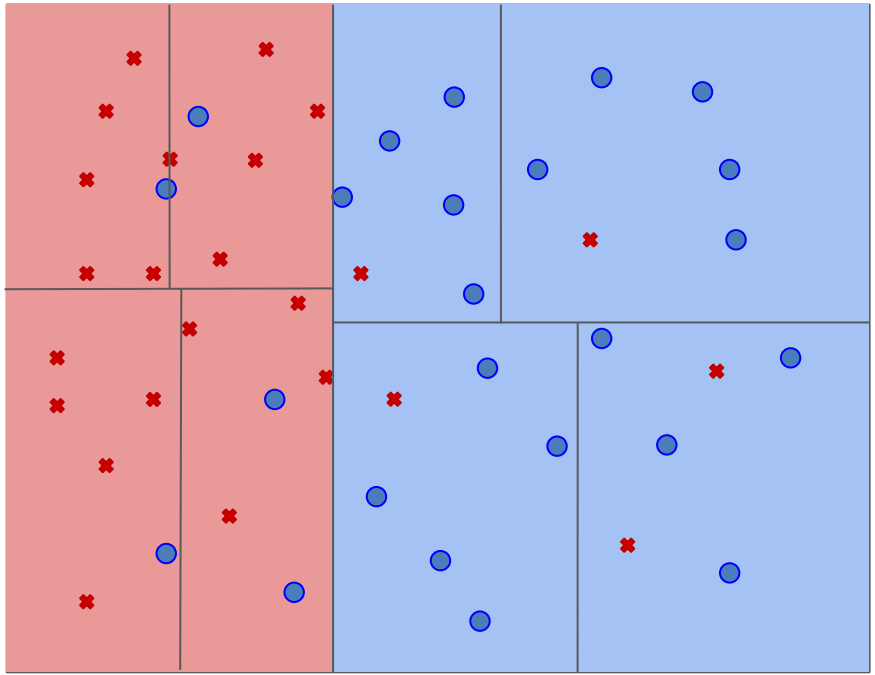
Both groups belong to class 1!





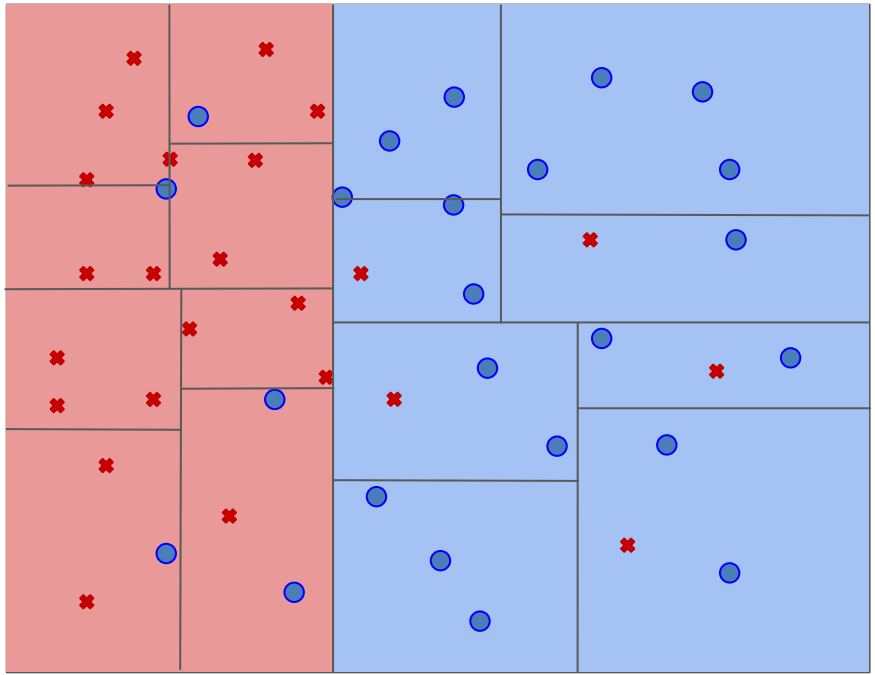
Third level

All splits still have the same class!



Fourth level

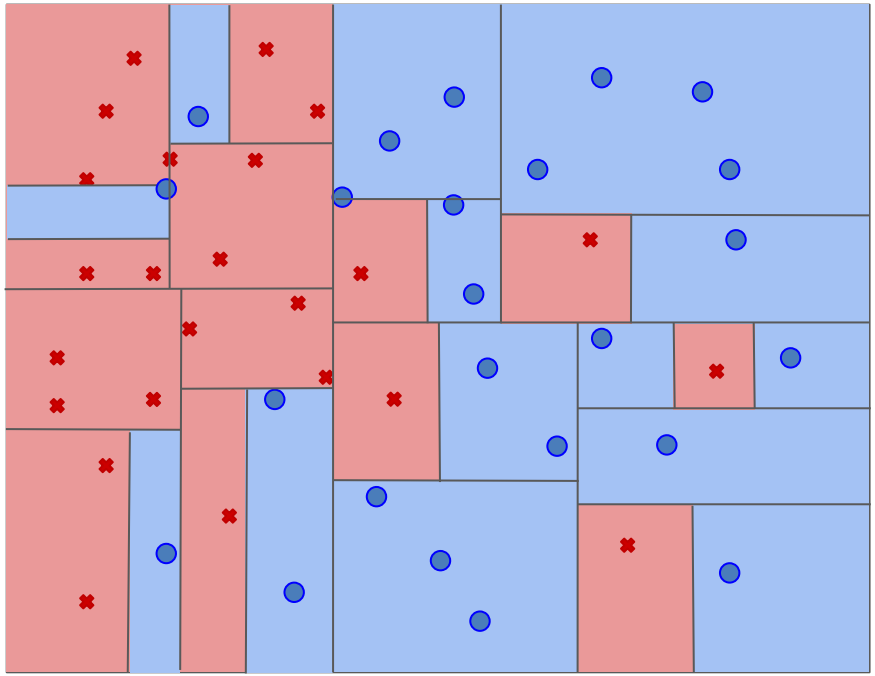
Some classes are all equal (the algorithm stops), like the one on the top right corner



The final tree

You can see why ID3 overfits: a lot of leaves have a single node in them!

The final depth of the tree is 6: with pruning or regularization, we would have stopped much earlier



Exercise: training a (small) random forest

We only train 3 trees (enough to get a majority vote)

Tree 1: features 1 and 2
Tree 2: features 2 and 3
Tree 3: features 1 and 3

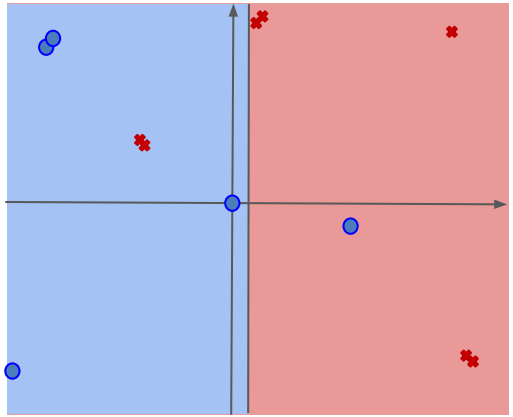
We stop after 2 levels

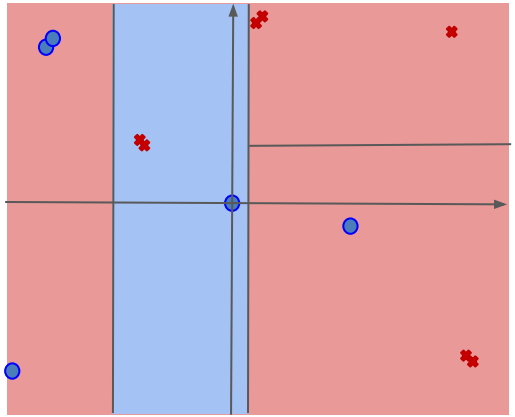
i	x_1	x_2	x_3	y
1	-1	0.8	0	1
2	-1.2	-0.9	0.3	1
3	-0.5	0.4	0.9	-1
4	0	0	1.2	1
5	0.1	1.1	0.2	-1
6	0.9	1.2	0.8	-1
7	0.6	-0.2	0.5	1
8	1.1	-0.8	1.2	-1



Training tree 1

We remove feature 2 and use bagging, then run ID3

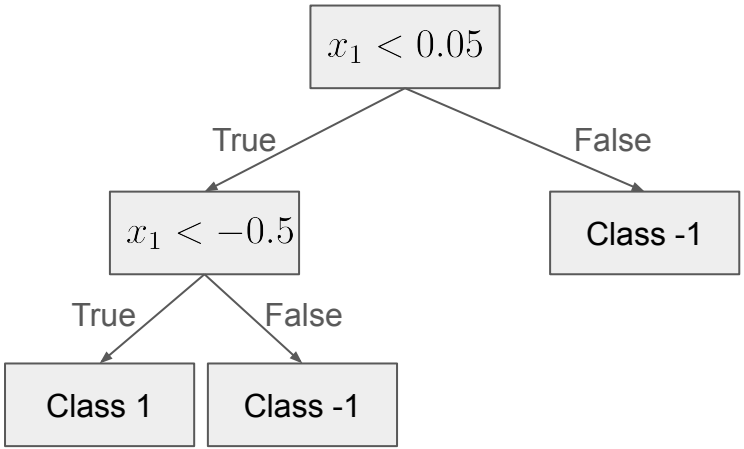




i	x_1	x_2	y
1	-1	0.8	1
2	-1.2	-0.9	1
4	0	0	1
5	0.1	1.1	-1
5	0.1	1.1	-1
6	0.9	1.2	-1
3	-0.5	0.4	-1
1	-1	0.8	1
7	0.6	-0.2	1
8	1.1	-0.8	-1
3	-0.5	0.4	-1
8	1.1	-0.8	-1

Training tree 1

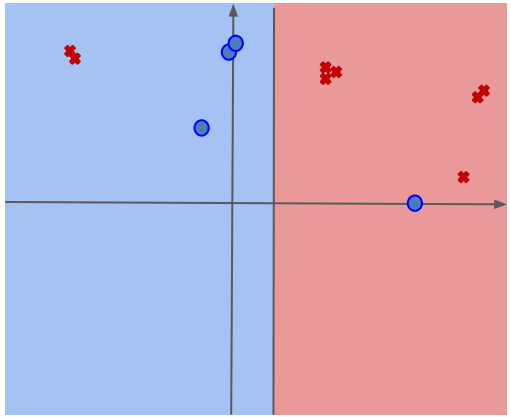
The final tree is:

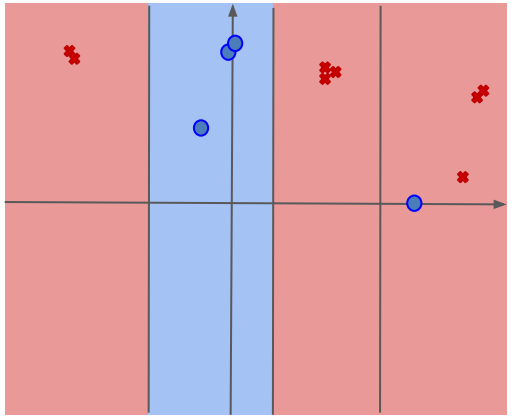


i	x_1	x_2	y
1	-1	0.8	1
2	-1.2	-0.9	1
4	0	0	1
5	0.1	1.1	-1
5	0.1	1.1	-1
6	0.9	1.2	-1
3	-0.5	0.4	-1
1	-1	0.8	1
7	0.6	-0.2	1
8	1.1	-0.8	-1
3	-0.5	0.4	-1
8	1.1	-0.8	-1

Training tree 2

We remove feature 1 and use bagging, then run ID3

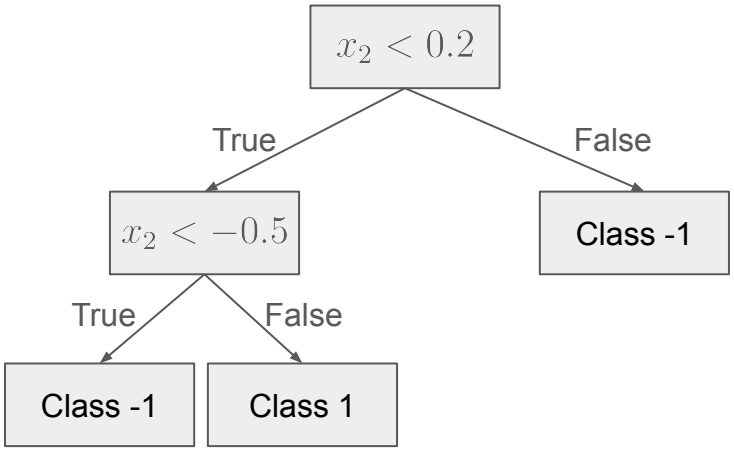




i	x_2	x_3	y
4	0	1.2	1
3	0.4	0.9	-1
3	0.4	0.9	-1
4	0	1.2	1
5	1.1	0.2	-1
6	1.2	0.8	-1
1	0.8	0	1
8	-0.8	1.2	-1
6	1.2	0.8	-1
7	-0.2	0.5	1
3	0.4	0.9	-1
8	-0.8	1.2	-1

Training tree 2

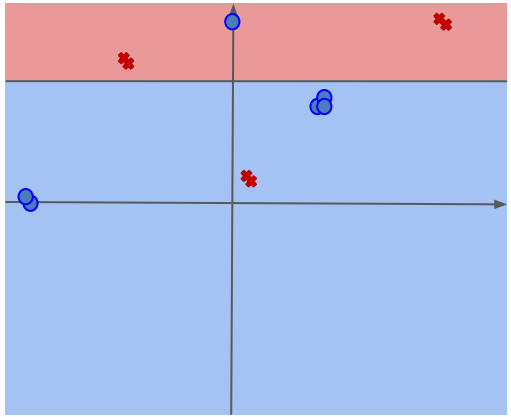
The final tree is:

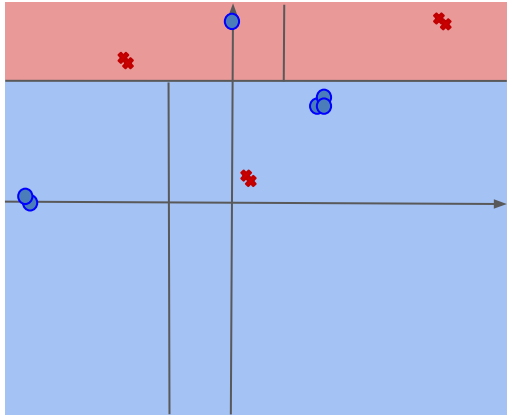


i	x_2	x_3	y
4	0	1.2	1
3	0.4	0.9	-1
3	0.4	0.9	-1
4	0	1.2	1
5	1.1	0.2	-1
6	1.2	0.8	-1
1	0.8	0	1
8	-0.8	1.2	-1
6	1.2	0.8	-1
7	-0.2	0.5	1
3	0.4	0.9	-1
8	-0.8	1.2	-1

Training tree 3

We remove feature 2 and use bagging, then run ID3

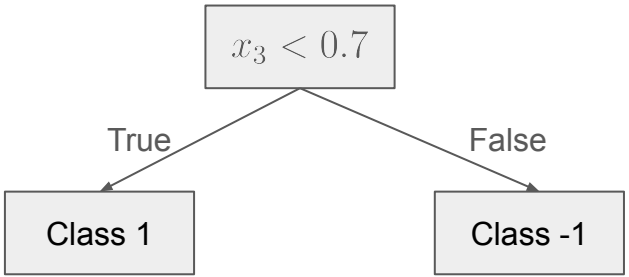




i	x_1	x_3	y
1	-1	0	1
3	-0.5	0.9	-1
3	-0.5	0.9	-1
4	0	1.2	1
5	0.1	0.2	-1
5	0.1	0.2	-1
7	0.6	0.5	1
8	1.1	1.2	-1
7	0.6	0.5	1
8	1.1	-0.8	-1
7	0.6	0.5	1
1	-1	0	1

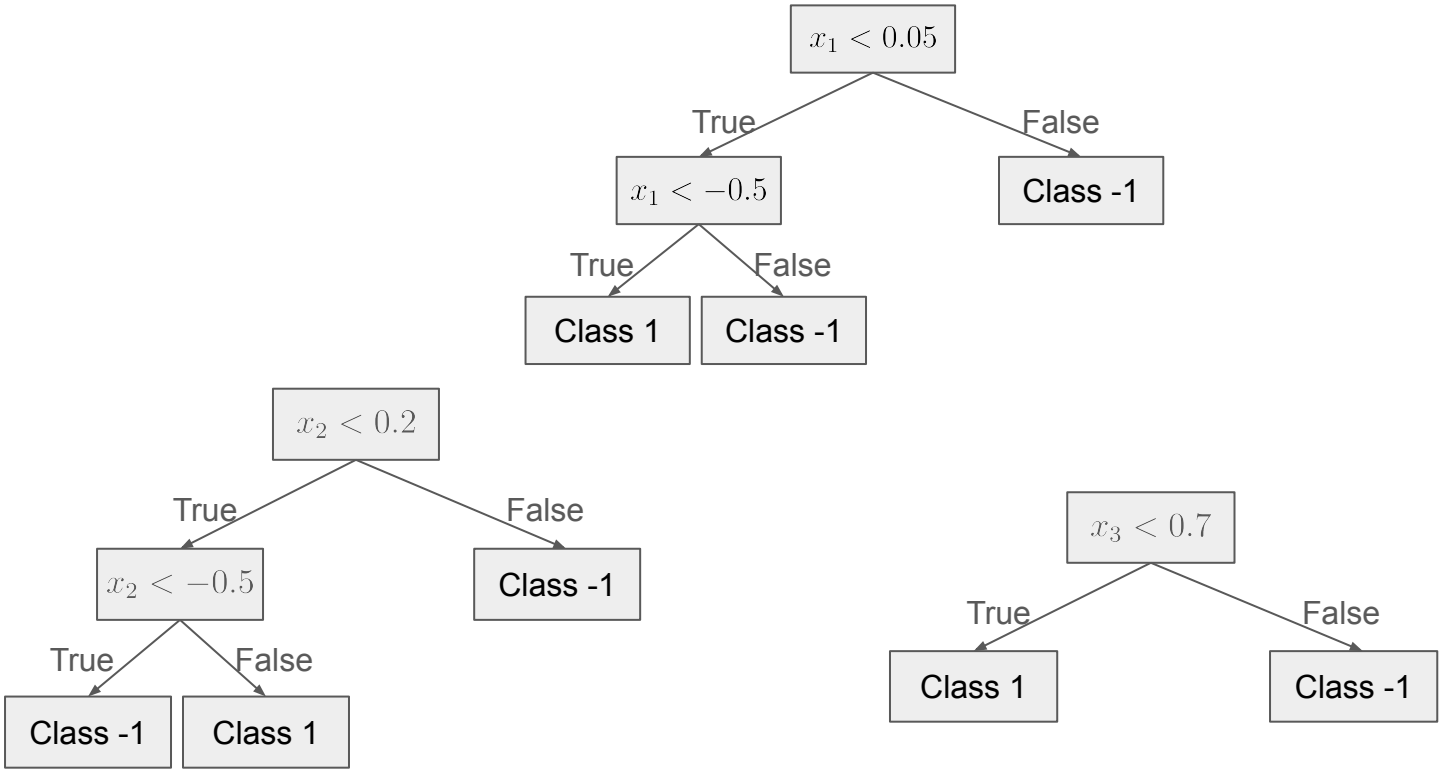
Training tree 3

The final tree is:



i	x_1	x_3	y
1	-1	0	1
3	-0.5	0.9	-1
3	-0.5	0.9	-1
4	0	1.2	1
5	0.1	0.2	-1
5	0.1	0.2	-1
7	0.6	0.5	1
8	1.1	1.2	-1
7	0.6	0.5	1
8	1.1	-0.8	-1
7	0.6	0.5	1
1	-1	0	1

The overall forest



The overall forest: results

i	x_1	x_2	x_3	y	Tree 1	Tree 2	Tree 3	**Forest**
1	-1	0.8	0	1	1	-1	1	**1**
2	-1.2	-0.9	0.3	1	1	-1	1	**1**
3	-0.5	0.4	0.9	-1	-1	-1	-1	**-1**
4	0	0	1.2	1	-1	1	-1	**-1**
5	0.1	1.1	0.2	-1	-1	-1	1	**-1**
6	0.9	1.2	0.8	-1	-1	-1	-1	**-1**
7	0.6	-0.2	0.5	1	-1	1	1	**1**
8	1.1	-0.8	1.2	-1	-1	-1	-1	**-1**